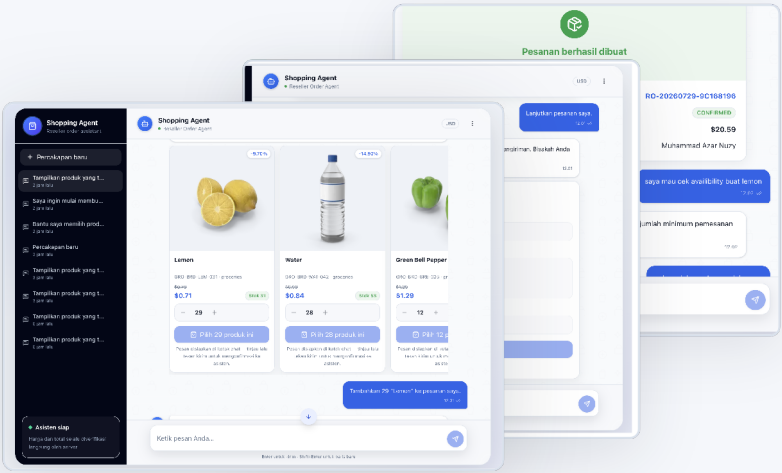
I built a conversational order agent while exploring practical full-stack AI engineering.

A technical case study on AI agents, tool calling, Langfuse tracing, evaluation, and transactional order safety.

</> Full-Stack Developer → AI Engineering



01 **Conversational UX**
Chat with context and state

02 **Tool calling**
15 typed agent tools

03 **Transactional safety**
Server-verified totals

04 **Langfuse tracing**
Privacy-aware observability

05 **Agent evaluation**
Deterministic case suite

06 **Monorepo boundaries**
Domain-driven packages

From product search to confirmed order — inside one conversation.

1 Search products
Ask in natural language.

Tampilkan produk yang tersedia, urutkan dari harga termurah.

Tentu, berikut adalah beberapa produk yang tersedia diurutkan dari harga termurah:

2 Select quantity
Choose how much you need.

$0.71	Stok 31	$0.84	Stok 53	$1.16	Stok 41
− 29 +		− 28 +		− 12 +	

Tambahkan 29 "Lemon" ke pesanan saya.

3 Build draft
Draft order is created with live totals.

Draft pesanan — Draft aktif
Versi 2 · 1 produk

Lemon — GRO-BRO-LEM-031 — Qty 29 — $20.59

Subtotal	$22.91
Diskon	−$2.32
Pengiriman	$0.00
Total pesanan	**$20.59**

4 Save customer data
Collect and validate delivery details.

ID Informasi pelanggan

Nama penerima * : John Doe

Nomor WhatsApp * : +62••••••••••

5 Review summary
Customer and order overview in one place.

Pelanggan
John Doe
+62••••••••••
Jl. Merdeka No. 10, Jakarta

Ringkasan pesanan

Subtotal	$22.91
Diskon	−$2.32
Total pesanan	**$20.59**

6 Create order
Order is created and ready to fulfill.

Pesanan berhasil dibuat
Pesanan Anda sudah kami terima.
RO-XXXXXXXX-XXXXXXX
CONFIRMED

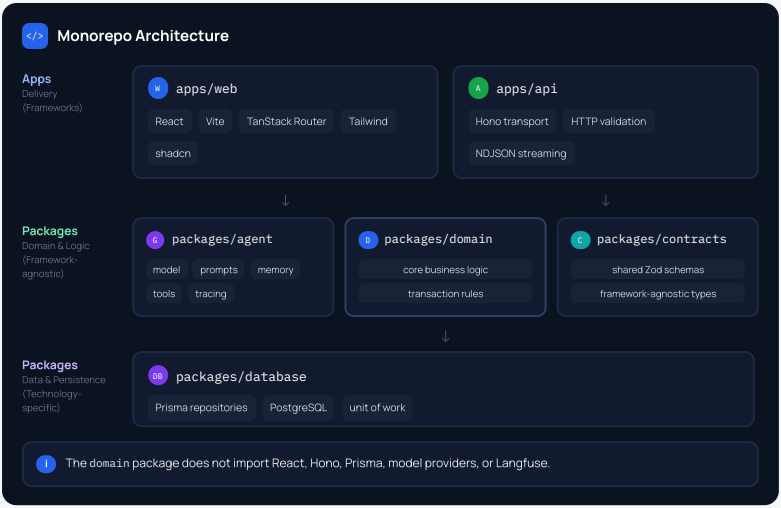
01 Client never sends trusted prices
Prices come from the domain service, not the client.

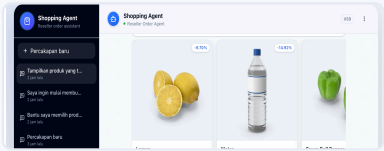
02 Totals are recalculated by the domain service
Discounts, shipping, and totals are server-verified.

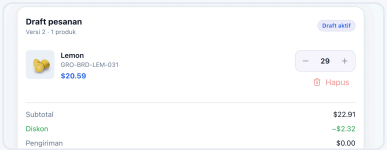
03 Order creation requires explicit confirmation
No order is created without a clear user confirmation.

One monorepo, with strict ownership between layers.

A full-stack TypeScript workspace designed for clear boundaries, reusable contracts, and safe transactions.







The agent reasons, but trusted services control the transaction.

A modular AI agent architecture with tool calling, dependency injection, and runtime guardrails.

M Model reasoning

T Transactional authority

C Catalog tools

- </> searchProducts
- </> getProductDetail
- </> checkProductAvailability
- </> recommendProducts
- </> getTopProducts
- </> listCategories

R

Agent

L Trusted runtime context

Isolated · Verified · Audited

- Session ID (server-side)
- Idempotency keys
- Audit logs

O Order tools

- </> getActiveDraft
- </> addDraftItem
- </> updateDraftItem
- </> removeDraftItem
- </> saveCustomerData
- </> validateDraft
- </> getOrderSummary
- </> confirmOrder
- </> getOrder

Engineering decisions

01 **Tools use dependency injection**

Loose coupling and swappable implementations.

02 **Tool inputs and outputs use Zod**

Strict schemas keep behavior predictable.

03 **Session ID stays in trusted context**

The model never sees session identifiers.

04 **No raw confirmation grants for the model**

Confirmation requires explicit user action.

05 **Prices and keys are not model inputs**

Prevents invented values and duplicates.

An AI response is not enough to create an order.

Order confirmation is enforced by runtime state, not by free-form text alone.

01
DRAFT
AI creates a draft

→

02
CUSTOMER_DATA_REQUIRED
Collect and validate customer data

→

03
READY_FOR_CONFIRMATION
All validations passed

→

04
EXPLICIT_USER_CONFIRMATION
User confirms intent explicitly

→

✓
CONFIRMED
Order created successfully

↓ ↓ ↓ ↓ ↓

? **AMBIGUOUS CONFIRMATION**
Not explicit, needs retry

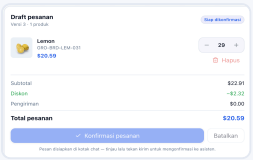
T **EXPIRED CONFIRMATION GRANT**
Time window expired

S **INVALID STOCK**
Stock changed or unavailable

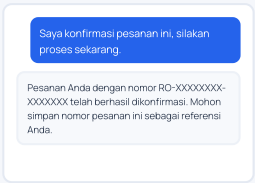
! **MINIMUM ORDER VIOLATION**
Does not meet minimum quantity

R **DUPLICATE RETRY**
Idempotency key already used

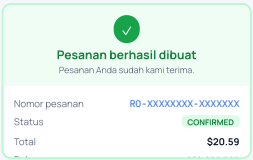
REVIEW DRAFT



EXPLICIT CONFIRMATION



ORDER SUCCESS



01
Confirmation grant is version-bound
Each draft version issues a unique grant with TTL.

02
`confirmOrder` uses a stable grant ID
The grant ID is required to advance to confirmation.

03
`POST /orders` requires an idempotency key
Prevents duplicate orders on retries.

04
Only one active draft is allowed per session
Prevents conflicting drafts and accidental duplicates.

State transitions are enforced server-side.
Free-form text alone can suggest — only explicit confirmation can commit.

AI behavior should be observable without exposing customer data.

Langfuse tracing was integrated as an engineering tool for observability, privacy, and failure-tolerant diagnostics.

`</>` `Trace: reseller-order-conversation-turn`

1. `msg` **Conversation turn** — User message captured with context and state
2. `llm` **Model generation** — LLM generates structured intent and plan
3. `</>` **Tool span** — Agent calls tool with sanitized arguments
4. `db` **Domain operation** — Business service executes and returns result
5. ✓ **Final response** — Assistant reply streamed back to the user

01 Session and user IDs become HMAC pseudonyms

02 Phone, email, address, authorization, and credentials are redacted

03 Nested tool payloads and errors are sanitized

04 Observer timeout never blocks order processing

05 CLI and API share one tracing factory

06 Traces flush on normal exit, errors, SIGINT, and SIGTERM

.env

```
LANGFUSE_ENABLED=true
LANGFUSE_TIMEOUT_MS=2000
TRACE_PSEUDONYM_KEY=••••••••
APP_RELEASE=git-sha
```

Lemon
$0.71

Water
$0.84

Green Bell Pepper
$1.16

01 **Conversational UX** — Natural chat with context

02 **Langfuse tracing** — Observability for LLM apps

03 **Tool calling** — Products, stock, drafts

04 **Evaluations** — Quality and safety

05 **Transactional safety** — Server-verified totals

06 **Production-minded** — Scalable architecture

I treated prompt changes like software changes.

Evaluation, regression checks, and failure analysis kept the agent grounded in business rules.

EV **Evaluation summary**
Deterministic evaluation results

Frozen prompt and model

15	**Evaluation cases** — Deterministic test cases
14	**Passed** — Success rate 93.3%

93.3% pass rate

TS	Tool selection	93.3% (14 / 15)	TA Tool arguments	93.3% (14 / 15)
FC	Factual correctness	100% (15 / 15)	BR Business rule compliance	100% (15 / 15)
NH	No hallucinated catalog facts	100% (15 / 15)	CS Cross-session safety	100% (15 / 15)

WF **Evaluation workflow**

1. **Frozen prompt and model** — Lock inputs and versions
2. **Run deterministic cases** — 15 curated cases
3. **Store raw results** — JSON artifacts
4. **Compare baseline vs candidate** — Diff metrics and traces
5. **Classify failure owner** — Tool / arg / data / rule
6. **Targeted regression test** — Fix → verify → lock

>_ Example evaluation commands — bash

```
pnpm eval -- --label baseline
pnpm eval -- --label candidate \
  --compare baseline/raw-results.json
pnpm eval -- --case eval-15-cross-session-order
```

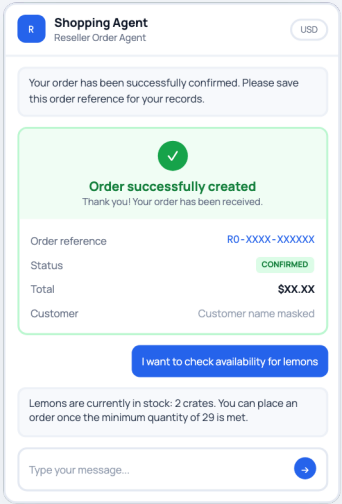
! **Remaining regression**

Cross-session order lookup still failed tool-selection expectations.

Owner: Tool selection · Priority: High

What I learned building an AI agent into a transactional product.

This project helped me connect full-stack engineering with practical AI agent architecture.



01 **The LLM should not own business truth**
Keep source-of-truth in verified systems.

02 **Tool design matters as much as prompt design**
Clear I/O contracts make agents reliable.

03 **Explicit confirmation needs runtime enforcement**
Don't rely on the model to be careful.

04 **Observability must be privacy-aware**
Log enough to debug, without exposing PII.

05 **Agent evaluation should run like regression testing**
Automate evals and catch regressions early.

Tech Stack

React Hono TypeScript Prisma PostgreSQL Zod Anvia Langfuse Vitest Docker

Conversational order agent
Natural chat to search, check stock, and order.

Production-minded stack
Typed APIs, tool calling, reliable data flow.

Observability and evaluations
Trace interactions, monitor quality, iterate.

Secure and reliable
Server-side verification and strong guardrails.

</> Building in public as a Full-Stack Developer learning AI Engineering.